

Reporte

Introducción

El código base proporcionado incluye un **NeuralAgent** que toma decisiones con un enfoque estrictamente *greedy*. Al evaluar únicamente el estado inmediato mediante una red neuronal y dos heurísticas básicas, el agente resulta tácticamente miope y es fácilmente acorralado. El objetivo de este proyecto es superar esta limitación desarrollando un **agente híbrido** (**AlphaBetaNeuralAgent**) que combine la intuición de la red con la planificación a largo plazo de la **Búsqueda Adversaria** (Minimax con poda AlphaBeta).

Para materializar este objetivo, el desarrollo se estructura en cuatro partes:

1. **Búsqueda Adversaria:** Integración del algoritmo Minimax con poda Alpha-Beta para evaluar ramificaciones futuras asumiendo respuestas óptimas de los adversarios.
2. **Mejora Heurística:** Expansión de la evaluación tradicional con nuevas heurísticas topológicas (huida proporcional y densidad de clústeres de comida) para puntuar los nodos terminales.
3. **Evaluación Híbrida:** Fusión matemática de la heurística clásica y la inferencia neuronal mediante un sistema de pesos dinámicos modulado por el nivel de amenaza.
4. **Validación Empírica:** Reentrenamiento del modelo con un *dataset* de alta calidad y análisis comparativo de rendimiento (*Winrate* y puntuación) en mapas estándar (**mediumClassic**) y modificados (**customMaze**).

En última instancia, el propósito didáctico de esta práctica es ilustrar cómo la integración del aprendizaje estadístico (redes neuronales) con métodos más tradicionales (búsqueda adversaria) permite diseñar arquitecturas híbridas capaces de superar las limitaciones inherentes de cada enfoque por separado.

Código fuente y control de versiones: [GitHub Repo](#).

Parte 1 - Nuevas Heurísticas

El código de Pacman contaba con una función de evaluación en **NeuralAgent**, la cual contaba con dos factores heurísticos básicos:

- **Factor 1:** Recompensa inversamente proporcional a la distancia al coco más cercano.
- **Factor 2:** Penalización fija de -200 si algún fantasma se encuentra a una distancia de Manhattan de ≤ 2 .

Para esta tarea, hemos añadido dos nuevos factores heurísticos adicionales a partir de los dos factores iniciales:

Heurística de Huida

```

for ghost_state in ghost_states:
    if ghost_state.scaredTimer == 0:
        ghost_pos = ghost_state.getPosition()
        ghost_distance = manhattanDistance(pacman_pos, ghost_pos)
        score -= 150.0 / (ghost_distance + 0.5)

```

Esta heurística aplica una penalización continua e inversamente proporcional a la distancia para cada fantasma que no esté asustado (independientemente de lo lejos que esté el fantasma).

La función: $150/(dist + 0.5)$ produce una curva hiperbólica que:

- Penaliza agresivamente distancias cortas.
- Penaliza suavemente distancias largas.

El factor heurístico dos hace que Pacman solo reaccione cuando los fantasmas ya están encima de él, pudiendo provocar situaciones donde se den encerronas. El factor heurístico que proponemos hace que Pacman sienta continuamente la presión de escapar y comience a alejarse de forma activa antes de que la situación se vuelva crítica.

Justificación: → La penalización solo se aplica a fantasmas **no** asustados (`scaredTimer == 0`). La idea principal es que, un Pacman que sobrevive más tiempo tiene más oportunidades de comer cocos y acumular puntuación. Huir activamente de los fantasmas evita situaciones críticas que no anticipa el factor 2. Al final, un Pacman que se dedica a huir puede acabar ganando una partida puesto que mientras que huye puede ir comiendo cocos, aunque no sea esta la intención principal.

- Peso elegido: 150. Un peso menor (ej. 50) no superaba la inercia del score base y Pacman seguía ignorando fantasmas lejanos; un peso mayor (ej. 300) hacía que Pacman huyera tanto que dejaba de comer. 150 equilibra supervivencia y comida.

Heurística de Food Clustering

```

if food:
    distances_to_food = sorted(
        manhattanDistance(pacman_pos, food_pos) for food_pos in food
    )
    nearest_cluster = distances_to_food[:5]
    avg_cluster_dist = sum(nearest_cluster) / len(nearest_cluster)
    score -= 0.4 * avg_cluster_dist

```

Esta heurística calcula la distancia media a los 5 cocos más cercanos y resta $0.4 * \text{esa media}$ a la puntuación. Si los cocos cercanos están agrupados, la distancia media es baja y la penalización es mínima, luego Pacman tenderá a limpiar esa zona. Por otro lado, si los cocos cercanos están dispersos, la penalización aumenta y Pacman no tenderá a moverse a esa zona.

Resumiendo:

- Penalización alta si los cocos cercanos están separados.

- Penalización suave si los cocos cercanos están cercanos.

El factor heurístico uno hace que pacman tienda al coco más cercano. Eso puede derivar en que pacman esté zigzagueando entre cocos aislados sin aprovechar agrupaciones. El factor que proponemos la da una visión a pacman sobre la densidad de la comida.

Justificación: → Acabar las partidas rápidamente minimiza el tiempo de exposición a los fantasmas y maximiza la puntuación final. Un Pacman capaz de limpiar las zonas densas primero es capaz de limpiar el tablero de forma más eficiente y rápida que uno que persigue cocos de uno en uno en extremos opuestos del mapa.

- Peso elegido: 0.4. Las distancias de Manhattan en el mapa *mediumClassic* oscilan entre 1 y aproximadamente 20, por lo que la contribución media de este factor es de entre -0.4 y -8 puntos, una influencia moderada que guía sin dominar la decisión.

Separación de Agentes

Uno de los objetivos era ser capaz de comparar ambas versiones (2 factores heurísticos vs 4 factores heurísticos) de forma sencilla. Para llevar a cabo este objetivo, las distintas versiones (2 vs 4 heurísticas) se han separado en dos clases (Agentes) independientes:

- **NeuralAgent:** Agente original. Contiene las dos heurísticas básicas. Se ha modificado eso si alguna cosa, como quitar los movimientos aleatorios iniciales con probabilidades decrecientes conforme avanza la partida, ya que hacían que Pacman se suicide contra los fantasmas, y cambiado la forma de tener en cuenta los valores de la red neuronal en la función de evaluación (*NeuralAgent:neuralEvaluation*), por razones explicadas en el código. Además se particionó la lógica de evaluación en una parte tradicional con las heurísticas (*traditionalEvaluation*) y de red neuronal (*neuralEvaluation*).
- **NeuralAgent2:** Hereda de **NeuralAgent** y sobrescribe únicamente *traditionalEvaluation* (las dos heurísticas originales) incluyendo nuestras heurísticas además de las originales, que las mantenemos en esta nueva implementación.

De esta forma, podemos ejecutar cada versión de forma independiente y comparar los resultados directamente:

Resultados Esperados

Se espera que **NeuralAgent2** consiga un comportamiento diferente al original: tender a evitar situaciones críticas y limpiar zonas con densidad alta de cocos. En cuanto a métricas, es posible que el *winrate* no mejore drásticamente respecto al original, ya que ganar con estos agentes es muy difícil.

Parte 2 - Entrenamiento de la Red Neuronal

Hemos desarrollado un agente híbrido (*HibridAgent*), con un *winrate* del 93.4% (500 ejecuciones), para obtener un dataset con partidas ganadas de alta calidad.

Estructura de HibridAgent

El diseño de *HibridAgent* se basa en una arquitectura de dos estados controlada por el riesgo. El agente alterna dinámicamente entre un modo de recolección rápida (Modo Pacífico) y una búsqueda adversaria

profunda (Modo Supervivencia). El objetivo de separar el comportamiento se debe a que Alpha-Beta a profundidad 4 es computacionalmente muy costoso. Si el entorno es seguro, es un desperdicio de CPU calcular las respuestas de los fantasmas y es mejor usar esos recursos para optimizar la ruta de recolección.

A continuación se detalla cada componente, su funcionamiento y su propósito exacto dentro del algoritmo:

1. Precomputación Topológica Estática (`_init_matrices`)

- **Qué hace:** En el primer turno de la partida, el agente escanea el laberinto y guarda en caché su topología completa. Esto incluye una matriz de adyacencia (distancias reales calculadas con BFS), la geometría de los pasillos, las intersecciones, y las rutas inerciales obligatorias que los fantasmas van a tomar en los túneles (ya que no pueden girar 180 grados).
- **Para qué sirve:** Evita el cuello de botella de tener que ejecutar búsquedas de caminos y otros computos costosos en cada nodo hoja del árbol Alpha-Beta. Al tener las distancias precalculadas, las consultas topológicas durante la expansión de nodos se resuelven en complejidad $O(1)$.

2. Evaluación de Riesgo Dinámica (`_is_state_safe`)

- **Qué hace:** Actúa como el selecto de modo del agente. En cada turno, mide la distancia real (teniendo en cuenta muros) a las amenazas (fantasmas). Define un estado como inseguro si hay algún fantasma activo a 3 pasos o menos, o si un fantasma que se acerca por pura inercia está a 5 pasos o menos. Por otro lado lo define como seguro cuando todos los fantasmas están asustados.
- **Para qué sirve:** Filtra cuándo es estrictamente necesario defenderse. Si no se cumplen estas condiciones, el estado se considera seguro y se desactiva el árbol Minimax para acelerar drásticamente el turno y centrarse en comer píldoras de manera eficiente.

3. Modo Pacífico (`_pacificEvaluationFunction`)

- **Qué hace:** Es una evaluación *Greedy* que prioriza limpiar el mapa. Utiliza el algoritmo de Prim para calcular el Árbol de Expansión Mínima (MST) de la comida restante, penaliza los granos que se quedan aislados en los extremos del laberinto para priorizarlos, y calcula el territorio seguro de Pacman mediante una partición de Voronoi (usando BFS).
- **Para qué sirve:** El cálculo del MST evita el comportamiento miope clásico donde Pacman va a la comida más cercana, pero deja otra aislada a la que luego le cuesta mucho llegar. Por otro lado, la partición de Voronoi asegura que Pacman mantenga una ruta de escape viable y no se meta en callejones sin salida, incluso cuando no haya un peligro inminente.

4. Modo Supervivencia (`evaluationFunction_local`)

- **Qué hace:** Es la heurística aplicada a los nodos hoja del árbol AlphaBeta. Emplea una penalización inverso-cuadrática ($1/d^2$) basada en la proximidad de los enemigos. Además, implementa lógicas de ruptura como el *Midgame Rush* (prioriza agresivamente ir a por una cápsula de poder si Pacman está más cerca de ella que cualquier fantasma) y el *Endgame Rush* (prioriza agresivamente ir a por las píldoras si solo quedan 10 o menos y si es seguro que va a poder recolectarlas antes de que le intercepte cualquier fantasma).
- **Para qué sirve:** Garantiza la supervivencia bajo presión. La penalización inverso-cuadrática fuerza a Pacman a hacer evasiones inmediatas y violentas a distancias cortas. Los modos *Rush* son parches

lógicos diseñados para evitar que Pacman se quede atascado esquivando fantasmas de forma infinita cuando tiene la oportunidad clara de comerse una cápsula o de ganar la partida.

5. Seleccionador de Acciones (**getAction**)

- **Qué hace:** Es la función orquestadora. Primero, calcula una acción de emergencia (evaluando a profundidad 1 cuál es el movimiento que lo aleja más de los fantasmas). Tras esto, usa el evaluador de riesgo para despachar la ejecución al Modo Pacífico o al Modo de Supervivencia dependiendo de si el estado actual se considera seguro o no.
- **Para qué sirve:** La acción de emergencia soluciona el problema clásico de la "muerte inevitable" en Minimax. Este sucede cuando el agente es acorralado y el árbol adversario determina que todas las ramas acaban en muerte segura (devolviendo $-\infty$ en todas las opciones), el bucle colapsa, paralizando a Pacman cuando este caso no es siquiera seguro la mayoría de las veces (ya que Minimax es pesimista). Al tener este salvavidas precalculado, Pacman intentará sobrevivir un turno más maximizando la distancia inmediata, en lugar de quedarse quieto o devolver una acción nula por puro pesimismo, aumentando sus probabilidades de sobrevivir.

Obtencion de Partidas

Luego, para obtener el dataset de partidas ganadas, hemos implementado un *script* de *bash* que nos permite ejecutar una cantidad total **N** de partidas. El *script* se encarga de ejecutar **C** partidas en paralelo, siendo **C** el numero de nucleos de la maquina (reportado por *nproc*), y de borrar automaticamente los CSV de salida de las partidas perdidas, asegurandonos que nos quedamos solo con partidas ganadoras.

Se puede ejecutar el *script* asi:

```
./scripts/megarun.sh N -p <NombreAgente> ...
```

Siendo **N** el numero de partidas a ejecutar, el agente a usar y ... cualquier otro parametro que se le quiera pasar al programa de pacman.

Como nuestro agente *HibridAgent*, es relativamente rapido (depth 4), y tiene un *winrate* elevado, hemos podido conseguir un *dataset* de alta calidad con 1729 partidas ganadas. Este dataset se encuentra en *./pacman_data/betterwins*. El otro *dataset* que conseguimos antes, ubicado en *./pacman_data/goodwins* son simplemente las partidas ganadas ejecutando *AlphaBetaRnAgent*, que tenia un *winrate* bajo del aproximadamente 15% y era muy lento por partida, con *depth* 5. Los *dataset* vienen comprimidos en formato **7z** porque sino ocupan demasiado (*betterwins*: ~450Mb, *goodiwns*: ~25Mb).

Cabe destacar, que para no obtener siempre la misma partida, se desactiva cualquier fijacion de semilla, haciendo las partidas realmente aleatorias. Es por esto, que si se reproduce cualquiera de las partidas de nuestro dataset, lo mas seguro es que se pierda, ya que como la semilla no va a ser la misma, Pacman va a actuar de la misma forma pero los fantasmas no, haciendo que muy probablemente pierda.

Cabe destacar que por defecto *megarun.sh* pone **AGENT_DEPTH**, la profundidad del agente, a 4. Esto se puede cambiar detro del *script* facilmente.

Cabe destacar que ambos *datasets* tienen solo partidas ganadas, es decir, un 100% de *winrate* en las partidas del *dataset*.

Archivado de los Modelos

Hemos ido guardando algunos de los modelos que hemos entrenado mientras íbamos probando cambiar los hiperparámetros del entrenamiento de la red neuronal. Estos se encuentran en `./models/own`, en carpetas numeradas en orden cronológico de menor (antes) a mayor (después). Dentro de cada carpeta se encuentra el modelo entrenado y un archivo `.info` con información de los hiperparámetros de entrenamiento del modelo, la *accuracy* final obtenida por el modelo, y los resultados medios de ejecutar 1000 partidas con el modelo usando *NeuralAgent*, además del *dataset* utilizado y cuantas partidas tenía en el momento. El modelo con mayor *accuracy* final obtenido ha sido el número 9, con un 90.36%, y sin overfitting, ya que estaban equiparado los *accuracies* de test y de validación.

Si se quiere cambiar el modelo activo, es decir, el que se encuentra en `./models/pacman_model.pth`, se puede hacer simplemente:

```
./scripts/model.sh N
```

Siendo **N** el nombre, o número en nuestro caso, de la carpeta donde se encuentra el modelo (solo mira en `./models/own`).

Parte 3 - AlphaBetaNeuralAgent

El agente *AlphaBetaNeuralAgent* integra la capacidad de búsqueda adversaria (Minimax con poda Alpha-Beta) con la intuición aprendida por la red neuronal y las heurísticas topológicas estructuradas en *NeuralAgent2*.

En lugar de utilizar la red neuronal para predecir el movimiento inmediato en el estado raíz, el agente despliega un árbol de búsqueda Alpha-Beta y delega la ejecución de la red neuronal a los nodos hoja. De este modo, la red no solo evalúa la situación actual, sino que puntúa las consecuencias futuras de los movimientos tras varios turnos de interacción con los fantasmas.

Cálculo del Final Score e Integración de Pesos

Para evaluar los nodos terminales del árbol de búsqueda, implementamos la función `evaluationFunction_local`, que calcula una suma ponderada entre el valor de las heurísticas tradicionales (los 4 factores de *NeuralAgent2*) y la predicción de la red neuronal.

La fórmula matemática aplicada en el código es una interpolación entre ambas evaluaciones:

$$FinalScore = Score_{tradicional} \cdot (1 - W_{dinamico}) + Score_{neuronal} \cdot W_{dinamico}$$

Donde $W_{dinamico}$ parte de un peso base configurable mediante la variable de entorno `NEURAL_WEIGHT` (por defecto establecido en 0.5).

Estrategia de Pesos Dinámicos

En lugar de mantener un peso estático durante toda la partida, implementamos una estrategia de pesos dinámicos modulada por el nivel de amenaza real. La idea principal es que la red neuronal es excelente para tácticas de evasión y supervivencia a corto plazo (situaciones de alta presión), pero las heurísticas tradicionales (como el factor de clusterización y la distancia de Manhattan) son matemáticamente superiores para optimizar la recolección de píldoras cuando el mapa está despejado.

El peso dinámico ($W_{dinamico}$) se calcula siguiendo este flujo de decisión en cada nodo evaluado:

1. **Amenaza nula (Recolección prioritaria):** Si no hay fantasmas activos en el mapa (todos están asustados), el peso de la red se fuerza a 0.0. El agente confía exclusivamente en las heurísticas para limpiar el mapa rápidamente.
2. **Cálculo de proximidad:** Si hay fantasmas activos, se calcula la distancia topológica real (usando BFS a través de `getMazeDistance`) al fantasma más cercano.
3. **Modulación por tramos:**
 - **Peligro inminente (Distancia ≤ 6 pasos):** El multiplicador es 1.0. El peso dinámico es igual al peso base (0.5). Se confía plenamente en el instinto de supervivencia de la red neuronal.
 - **Zona segura (Distancia ≥ 12 pasos):** El multiplicador es 0.0. La amenaza está demasiado lejos, por lo que la red se ignora en favor de la recolección heurística.
 - **Zona de transición (Distancia entre 6 y 12 pasos):** Se aplica una interpolación lineal para suavizar la transición de comportamientos, calculada como $Factor = (12 - Distancia)/6$. Por ejemplo, a 9 pasos de distancia, el factor es 0.5, asignando la mitad del peso base a la red neuronal.

Prevención de Colapso (Fallback Heurístico)

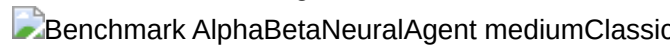
Finalmente, la integración incluye un mecanismo de seguridad en la función raíz `getAction` para solucionar el problema del "pesimismo excesivo" de Minimax del que hablamos antes. Si el agente es acorralado y el árbol proyecta una muerte inevitable en todas las ramas posibles (devolviendo $-\infty$ globalmente), el bucle colapsaría intentando ejecutar una acción nula.

Para evitarlo el agente calcula previamente un movimiento *Greedy* (a profundidad 1) utilizando la función combinada. Si Alpha-Beta falla, el agente devuelve esta acción de *fallback*, asegurando que Pacman ejecute el movimiento que maximice su supervivencia inmediata en lugar de quedarse inmóvil rendido a su destino.

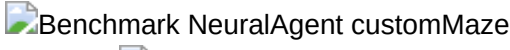
Parte 4 - Resultados

A continuación se muestran las capturas de ejecución y la tabla comparativa con los resultados obtenidos al someter a los distintos agentes a 100 ejecuciones.







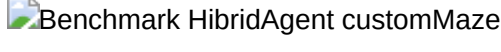



Tabla Comparativa de Rendimiento

Configuración	mediumClassic (Score / Win Rate)	customMaze (Score / Win Rate)
NeuralAgent (Original)	156.58 / 4%	134.81 / 0%
NeuralAgent2 (+ Heurísticas)	273.66 / 9%	150.94 / 1%
AlphaBetaNeuralAgent (Final)	259.94 / 2%	402.28 / 3%
HibridAgent (Pura Búsqueda/Referencia)	1507.48 / 97%	1408.45 / 90%

Análisis de los Resultados

1. Impacto de las nuevas heurísticas (NeuralAgent vs NeuralAgent2) La adición de la huida proporcional y la clusterización de comida mejora de forma objetiva el rendimiento base. En **mediumClassic**, el *win rate* se duplica (del 4% al 9%) y la puntuación media sube más de 100 puntos. Las heurísticas evitan que Pacman se quede atascado en bucles de persecución y optimizan la limpieza del mapa, demostrando que dotar al agente de conocimiento topológico directo es más efectivo que depender únicamente de la inferencia de la red en crudo.

2. Impacto de la Búsqueda Adversaria (AlphaBetaNeuralAgent) La integración de Alpha-Beta arroja resultados mixtos. En el mapa de entrenamiento (**mediumClassic**), el *win rate* sufre una ligera caída debido a que la evaluación de los nodos hoja está fuertemente sesgada por las predicciones de una red neuronal que, estadísticamente, tiene un margen de error elevado. Sin embargo, en **customMaze**, la búsqueda en profundidad demuestra su valor: aunque el *win rate* sigue siendo bajo, la puntuación media se dispara a 402.28 puntos. Esto indica que el árbol Minimax permite a Pacman sobrevivir mucho más tiempo en entornos desconocidos al prever amenazas, algo de lo que los agentes puramente reactivos son incapaces.

3. Generalización a nuevos mapas (customMaze) Los tres agentes basados en redes neuronales sufren un colapso de rendimiento al cambiar de topología. Esto evidencia un claro sobreajuste (*overfitting*) espacial: la red ha memorizado patrones geométricos y secuencias de movimiento del mapa **mediumClassic**. Al abrirse nuevos pasillos en **customMaze**, la distribución espacial de los inputs cambia radicalmente, inutilizando la experiencia aprendida.

¿Por qué **HibridAgent** es inmensamente superior?

El contraste entre **HibridAgent** (90-97% de victorias) y las variantes neuronales expone una realidad algorítmica fundamental: **Pacman es un entorno determinista, discreto y de información perfecta.**

En este tipo de problemas, los métodos de IA clásica fundamentados en la Teoría de Juegos y la Teoría de Grafos (Minimax, Alpha-Beta, MST, Voronoi) siempre van a dominar. **HibridAgent** calcula distancias reales y proyecta simulaciones matemáticas exactas de lo que va a ocurrir. Sabe con certeza absoluta si un fantasma le va a alcanzar o no.

Por el contrario, el enfoque neuronal utilizado en esta práctica emplea **Aprendizaje Supervisado** para imitar a un experto. La red intenta mapear una matriz de entrada a una distribución de probabilidad de acciones sin entender las reglas del juego. No sabe qué es una pared o qué significa morir; solo busca correlaciones estadísticas en los píxeles. En un laberinto donde moverse un píxel a la derecha significa la salvación y a la izquierda la muerte inmediata, una ligera imprecisión estadística en la inferencia resulta letal.

¿Cuándo es brilla usar Redes Neuronales?

Aplicar redes neuronales puras para la navegación a bajo nivel en Pacman es como usar una excavadora para clavar un clavo. Sin embargo, las redes neuronales sobresalen resolviendo problemas donde la IA clásica fracasa estrepitosamente:

1. **Espacios de estado inabarcables o continuos:** En juegos como Go (donde el árbol de búsqueda es mayor que el número de átomos del universo) o en la conducción autónoma, es imposible expandir un árbol Minimax hasta el final. Las redes neuronales (como en AlphaGo) resuelven esto actuando como "estimadores de valor", recortando el árbol al evaluar de un vistazo la calidad abstracta de un estado.
2. **Información imperfecta:** En entornos como el Póker o StarCraft, donde existe niebla de guerra o variables ocultas, la heurística determinista colapsa. Las redes neuronales pueden aprender a inferir probabilidades y patrones a partir de información incompleta.
3. **Manejo de ruido y percepción:** Si Pacman no se jugase sobre una matriz numérica perfecta, sino a través de los píxeles crudos de la pantalla con ruido visual, una red neuronal convolucional (CNN) sería indispensable para la extracción de características antes de que cualquier algoritmo de búsqueda pudiese tomar una decisión.

Cabe destacar que en el caso de *HibridAgent* con *customMaze* solo se han ejecutado 20 partidas, no 100. Esto se debe a que en este mapa con tanto espacio abierto este agente es considerablemente mas lento.

Conclusiones

El análisis comparativo de los distintos agentes desarrollados en esta práctica permite extraer conclusiones fundamentales sobre la aplicabilidad de técnicas de Inteligencia Artificial en entornos discretos, deterministas y de información perfecta:

1. **Ineficiencia del Aprendizaje Supervisado puro:** Dependier exclusivamente de la inferencia estadística para la navegación a bajo nivel resulta subóptimo. Los agentes puramente neuronales mostraron una severa miopía táctica y un claro sobreajuste (*overfitting*) a la topología de entrenamiento (*mediumClassic*), colapsando su rendimiento en escenarios desconocidos (*customMaze*).
2. **Valor de la Búsqueda Adversaria y el Conocimiento de Dominio:** La integración del algoritmo Minimax con poda Alpha-Beta, junto con heurísticas topológicas explícitas (huida proporcional y clusterización), dotó al agente de capacidad de anticipación. Esto mejoró sustancialmente la supervivencia y el proceso de toma de decisiones frente al enfoque puramente reactivo.
3. **Dominio de la IA Simbólica:** El rendimiento diferencial del *HibridAgent* (90-97% de victorias) constata que, en laberintos estructurados por reglas exactas sin variables ocultas, la Teoría de Juegos y el análisis de grafos superan con creces a la aproximación neuronal.
4. **Justificación de Agentes Híbridos:** Aunque el *AlphaBetaNeuralAgent* demuestra que es posible fusionar con éxito la planificación a largo plazo con la intuición aprendida, el uso de redes neuronales en este problema específico resulta excesivo. Su implementación encontraría verdadera justificación en variantes del juego con espacios de estado continuos, percepción basada en píxeles crudos o entornos con información imperfecta (niebla de guerra).

Comandos del Video

Aqui se encuentran los comandos necesarios para reproducir las partidas vistas en los videos.

Hace falta tener en cuenta que se pasan todos los archivos de codigo ya que muchos de ellos tienen alguna pequeña modificacion para responder a la variable de entorno *SEED* y fijar su semilla en consecuencia. Sin estos cambios las partidas no son reproducibles.

Se asume que el mapa *mediumClassic* sigue siendo el mapa por defecto, sino solo hace falta añadir -
`l mediumClassic`.

AlphaBetaNeuralAgent

- Con el mapa normal:

```
SEED=719 AGENT_DEPTH=4 ./scripts/run.sh -p AlphaBetaNeuralAgent
```

- Con el nuevo mapa:

```
SEED=719 AGENT_DEPTH=4 ./scripts/run.sh -p AlphaBetaNeuralAgent -l  
customMaze
```

HibridAgent

- Con el mapa normal:

```
SEED=719 AGENT_DEPTH=4 ./scripts/run.sh -p HibridAgent
```

- Con el nuevo mapa:

```
SEED=719 AGENT_DEPTH=4 ./scripts/run.sh -p HibridAgent -l customMaze
```